📁 Files (java.nio.file.Files)

Acción	Código	Comentarios
Leer texto	`String txt = Files.readString(ruta, StandardCharsets.UTF_8);`	Todo el archivo → `String`.
Escribir texto	`Files.writeString(ruta, txt, StandardCharsets.UTF_8);`	Crea o sobrescribe.
Leer bytes	`byte[] datos = Files.readAllBytes(ruta);`	Todo el archivo → `byte[]`.
Escribir bytes	`Files.write(ruta, datos);`	Crea o sobrescribe.
Leer líneas	`List<String> lineas = Files.readAllLines(ruta, StandardCharsets.UTF_8);`	Una `String` por línea.
Escribir líneas	`Files.write(ruta, lineas, StandardCharsets.UTF_8);`	Añade salto de línea entre elementos.
Añadir (append)	`Files.writeString(ruta, txt, StandardOpenOption.APPEND);`	No sobrescribe.
Copiar	`Files.copy(origen, destino, StandardCopyOption.REPLACE_EXISTING);`	Sobrescribe si existe.
Mover	`Files.move(origen, destino, StandardCopyOption.REPLACE_EXISTING);`	Renombra o mueve.
Borrar	`Files.delete(ruta);`	Falla si no existe; `deleteIfExists` no.
Existe	`Files.exists(ruta);`	`true` / `false`.
Tamaño	`long bytes = Files.size(ruta);`	Tamaño en bytes.
Crear directorio	`Files.createDirectories(ruta);`	Crea toda la ruta.
Listar directorio	`try (Stream<Path> s = Files.list(ruta)) { ... }`	Stream de entradas; cerrar con try-with-resources.
Recorrer árbol	`try (Stream<Path> s = Files.walk(ruta)) { ... }`	Recursivo; cerrar con try-with-resources.

Lectura por bloques (archivos grandes)

> `Files.readAllBytes` *carga todo en memoria; para archivos grandes, leer por bloques.*

Leer

```
try (InputStream entrada = Files.newInputStream(ruta)) {
   ByteArrayOutputStream acum = new ByteArrayOutputStream();
   byte[] buffer = new byte[1024];

   int n;
   while ((n = entrada.read(buffer)) != -1) {
      acum.write(buffer, 0, n);
   }

   byte[] datos = acum.toByteArray();
}
```

Escribir

```
try (OutputStream salida = Files.newOutputStream(ruta)) {
   byte[] datos = msg.getBytes(StandardCharsets.UTF_8);
   salida.write(datos);
   salida.flush();
}
```

📡 UDP (DatagramSocket)

Acción	Cliente	Servidor	Comentarios
Obtener dirección	`InetAddress.getByName(host)`	Del paquete recibido	`host`: hostname o IP, p. ej. `"192.168.1.100"`.
Crear socket	`new DatagramSocket()`	`new DatagramSocket(puerto)`	Cliente: puerto efímero; servidor: fijo.
Preparar datos	`String msg = "Hola";` `byte[] datos = msg.getBytes(StandardCharsets.UTF_8);`	Igual	`msg` es el mensaje de texto a enviar; `datos` son sus bytes en UTF-8.
Crear paquete (envío)	`new DatagramPacket(datos, datos.length, serverAddress, serverPort)`	`new DatagramPacket(datos, datos.length, clientAddress, clientPort)`	Datos + destino.
Enviar	`socket.send(paquete)`	`socket.send(paquete)`	UDP no garantiza entrega ni orden.
Preparar recepción	`new DatagramPacket(buffer, buffer.length)`	Igual	`byte[] buffer = new byte[1024];`
Recibir	`socket.receive(paquete)`	`socket.receive(paquete)`	Bloqueante. Timeout: `socket.setSoTimeout(ms)`.
Extraer datos	`new String(p.getData(), p.getOffset(), p.getLength(), StandardCharsets.UTF_8)`	Igual; además, `p.getAddress()` y `p.getPort()`	`getLength()` indica los bytes recibidos.
Cerrar	`socket.close()`	`socket.close()`	Libera el socket.

Fragmentación

> *Cada datagrama lleva un byte de cabecera:* `0` *= quedan fragmentos,* `1` *= último. Payload útil = 1023 bytes (buffer 1024 − 1).*

Cliente

```
byte[] datos = msg.getBytes(StandardCharsets.UTF_8);

for (int i = 0; i < datos.length; i += 1023) {
   int n = Math.min(1023, datos.length - i);
   byte[] frag = new byte[n + 1];
   frag[0] = (byte) (i + n == datos.length ? 1 : 0);
   System.arraycopy(datos, i, frag, 1, n);
   socket.send(new DatagramPacket(frag, frag.length, addr, puerto));
}
```

Servidor

```
ByteArrayOutputStream acum = new ByteArrayOutputStream();
byte[] buffer = new byte[1024];

do {
   DatagramPacket paquete = new DatagramPacket(buffer, buffer.length);
   socket.receive(paquete);
   acum.write(buffer, paquete.getOffset() + 1, paquete.getLength() - 1);
} while (buffer[paquete.getOffset()] != 1);

String msg = acum.toString(StandardCharsets.UTF_8);
```

📡 TCP (Socket / ServerSocket)

Acción	Cliente	Servidor	Comentarios
Obtener dirección	`InetAddress.getByName(host)`	—	`host`: hostname o IP, p. ej. `"192.168.1.100"`.
Crear socket	`new Socket(host, puerto)`	`new ServerSocket(puerto)`	Cliente: conecta al servidor; servidor: escucha en puerto fijo.
Aceptar conexión	—	`Socket socket = serverSocket.accept()`	Bloqueante; devuelve un socket por cliente.
Preparar datos	`String msg = "Hola";` `byte[] datos = msg.getBytes(StandardCharsets.UTF_8);`	Igual	`msg` es el mensaje de texto a enviar; `datos` son sus bytes en UTF-8.
Enviar	`salida.write(datos)` `salida.flush()`	`salida.write(datos)` `salida.flush()`	`salida = socket.getOutputStream()`. TCP garantiza entrega y orden.
Preparar recepción	`byte[] buffer = new byte[1024];` `entrada = socket.getInputStream();`	Igual	`entrada = socket.getInputStream()`.
Recibir	`int n = entrada.read(buffer);`	`int n = entrada.read(buffer);`	Bloqueante; `n == -1` indica fin de stream. Timeout: `socket.setSoTimeout(ms)`.
Extraer datos	`new String(buffer, 0, n, StandardCharsets.UTF_8)`	Igual	`n` indica los bytes leídos.
Cerrar	`socket.close()`	`socket.close()` `serverSocket.close()`	Libera el socket (y el servidor, el de escucha).

Mensajes largos

> *TCP es un* ***stream de bytes****: no hay datagramas ni fragmentación manual.* `write` *y* `read` *trabajan sobre un flujo continuo; el mensaje se reconstruye leyendo hasta completar el tamaño esperado o hasta* `-1`.

Cliente

```
OutputStream salida = socket.getOutputStream();

byte[] datos = msg.getBytes(StandardCharsets.UTF_8);
salida.write(datos);
salida.flush();
```

Servidor

```
InputStream entrada = socket.getInputStream();
ByteArrayOutputStream acum = new ByteArrayOutputStream();
byte[] buffer = new byte[1024];

int n;
while ((n = entrada.read(buffer)) != -1) {
   acum.write(buffer, 0, n);
}

String msg = acum.toString(StandardCharsets.UTF_8);
```